

COLÉGIO CÍVICO-MILITAR — HORIZONTE

Curso Técnico em Análise e Desenvolvimento de Sistemas

LabTech

Sistema Digital de Gestão de Empréstimos do Laboratório de Informática

DOCUMENTAÇÃO TÉCNICA COMPLETA

Kauã Tatim · Gabriely · Igor Rodrigues · Guilherme · Jean · José

Horizonte, Ceará — 2026

SUMÁRIO

- 1. Introdução
 - 1.1 Apresentação do Problema
 - 1.2 Solução Proposta
 - 1.3 Objetivos
 - 1.4 Escopo do Sistema
- 2. Requisitos do Sistema
 - 2.1 Requisitos Funcionais
 - 2.2 Requisitos Não Funcionais
- 3. Análise e Modelagem
 - 3.1 Perfis de Usuário
 - 3.2 Diagrama de Caso de Uso
 - 3.3 Diagrama de Classes
 - 3.4 Diagrama de Sequência
- 4. Banco de Dados
 - 4.1 Modelagem Lógica
 - 4.2 Descrição das Tabelas
 - 4.3 Triggers e Views
- 5. Arquitetura do Backend
 - 5.1 Tecnologias Utilizadas
 - 5.2 Estrutura de Módulos
 - 5.3 Rotas da API REST
 - 5.4 Autenticação e Autorização
- 6. Protótipo das Telas (Figma)
 - 6.1 Fluxo de Navegação
 - 6.2 Telas por Perfil
- 7. Cronograma

8. Divisão de Responsabilidades

9. Glossário

10. Referências

1. Introdução

1.1 Apresentação do Problema

O Laboratório de Informática do Colégio Cívico-Militar de Horizonte não possui um sistema formalizado para controlar o empréstimo de tablets e equipamentos. Os registros são realizados de forma manual — em papel ou planilhas —, o que resulta em:

- Ausência de rastreabilidade: impossível saber com quem um tablet está, por quanto tempo e em qual estado foi devolvido
- Perda de equipamentos por falta de controle e cobrança
- Retrabalho administrativo para professores e coordenação pedagógica
- Impossibilidade de visualizar disponibilidade em tempo real

1.2 Solução Proposta

O LabTech é um sistema web de gestão de empréstimos de equipamentos do laboratório de informática, que digitaliza completamente o processo: o usuário solicita um item, um responsável aprova ou rejeita, o sistema registra a devolução e mantém o status atualizado em tempo real. Todo o histórico de movimentações fica armazenado e acessível por perfil.

1.3 Objetivos

Objetivo Geral

Desenvolver um sistema web para controle e registro de empréstimos de tablets do laboratório de informática, com acesso diferenciado por perfil de usuário.

Objetivos Específicos

- Digitalizar o processo de solicitação, aprovação e devolução de equipamentos
- Oferecer visão em tempo real do status de cada tablet (disponível, emprestado, manutenção)
- Implementar controle de acesso por perfil (Aluno, Professor, Pedagogo, Diretor)
- Registrar histórico completo de todas as movimentações de equipamentos
- Restringir cadastro e edição de itens e usuários ao perfil Diretor
- Permitir que professores e pedagogos aprovelem e registrem devoluções

1.4 Escopo do Sistema

O sistema abrange exclusivamente a gestão de empréstimos de tablets do laboratório de informática. Outras categorias de equipamentos (mouse, teclado, notebook) e o módulo de Calendário de Eventos estão fora do escopo do MVP atual.

2. Requisitos do Sistema

2.1 Requisitos Funcionais

ID	Descrição	Perfil	Prioridade
RF-01	O sistema deve permitir login com e-mail e senha	Todos	Alta
RF-02	O sistema deve permitir cadastro de novos usuários	Diretor	Alta
RF-03	O sistema deve permitir consulta de tablets disponíveis	Todos	Alta
RF-04	O sistema deve permitir solicitação de empréstimo de tablet	Aluno / Professor	Alta
RF-05	O sistema deve registrar data de retirada e devolução prevista	Todos	Alta
RF-06	O sistema deve permitir aprovação de solicitações pendentes	Prof / Ped / Diretor	Alta
RF-07	O sistema deve permitir rejeição de solicitações	Prof / Ped / Diretor	Alta
RF-08	O sistema deve registrar devoluções de tablets	Prof / Ped / Diretor	Alta
RF-09	O status do tablet deve ser atualizado automaticamente	Sistema	Alta
RF-10	O sistema deve exibir histórico de empréstimos	Prof / Ped / Diretor	Média
RF-11	O sistema deve permitir cadastro de tablets	Diretor	Alta
RF-12	O sistema deve permitir edição de tablets	Diretor	Média

RF-13	O sistema deve exibir dashboard com visão geral	Todos	Alta
RF-14	O sistema deve exibir lista de usuários cadastrados	Ped / Diretor	Média
RF-15	O sistema deve gerar token JWT ao autenticar	Sistema	Alta

2.2 Requisitos Não Funcionais

ID	Descrição	Categoria
RNF-01	Senhas devem ser armazenadas com hash bcrypt (mínimo 10 rounds)	Segurança
RNF-02	Todas as rotas protegidas devem exigir token JWT válido	Segurança
RNF-03	O sistema deve retornar resposta em menos de 2 segundos	Performance
RNF-04	A interface deve ser responsiva e funcionar em diferentes resoluções	Usabilidade
RNF-05	O banco de dados deve suportar integridade referencial via FK	Confiabilidade
RNF-06	O código deve seguir os padrões de arquitetura do NestJS	Manutenibilidade
RNF-07	O sistema deve funcionar nos navegadores Chrome e Firefox atuais	Compatibilidade

3. Análise e Modelagem

3.1 Perfis de Usuário

O sistema possui quatro perfis com permissões distintas, implementados via role-based access control (RBAC) no backend:

Perfil	Atributos Específicos	Permissões
Aluno	curso, matrícula	Solicitar empréstimo · Consultar tablets · Ver seus próprios empréstimos
Professor	departamento	Solicitar empréstimo · Aprovar/Rejeitar · Registrar devolução · Ver histórico
Pedagogo	área de atuação	Aprovar/Rejeitar · Registrar devolução · Ver usuários · Ver histórico completo
Diretor	—	Acesso total: cadastrar itens e usuários · Todas as permissões anteriores

3.2 Diagrama de Caso de Uso

Atores

- Aluno — usuário final que solicita tablets para uso em atividades escolares
- Professor — solicita e aprova/rejeita empréstimos da sua turma
- Pedagogo — supervisiona empréstimos e tem acesso ao histórico completo
- Diretor — administrador do sistema com controle total

Casos de Uso

Caso de Uso	Ator(es)	Pré-condição	Resultado
UC-01: Autenticar via login	Todos	Usuário cadastrado no sistema	Sessão JWT ativa
UC-02: Consultar tablets disponíveis	Todos	Autenticado	Lista de tablets com status
UC-03: Solicitar empréstimo	Aluno / Professor	Tablet disponível	Empréstimo PENDENTE criado
UC-04: Aprovar empréstimo	Prof / Ped / Diretor	Empréstimo PENDENTE	Status → APROVADO, tablet → EMPRESTADO
UC-05: Rejeitar empréstimo	Prof / Ped / Diretor	Empréstimo PENDENTE	Status → REJEITADO, tablet → DISPONÍVEL
UC-06: Registrar devolução	Prof / Ped / Diretor	Empréstimo APROVADO	Status → DEVOLVIDO, tablet → DISPONÍVEL
UC-07: Cadastrar tablet	Diretor	Autenticado como Diretor	Novo tablet no sistema
UC-08: Editar tablet	Diretor	Tablet cadastrado	Dados do tablet atualizados
UC-09: Cadastrar usuário	Diretor	Autenticado como Diretor	Novo usuário no sistema
UC-10: Visualizar histórico	Prof / Ped / Diretor	Autenticado	Lista de todos os empréstimos
UC-11: Gerenciar permissões	Diretor	Usuário existente	Perfil do usuário alterado

3.3 Diagrama de Classes

Classes Principais

Classe	Tipo	Atributos	Métodos
Usuario	Abstract	id: Long nome: String emailGoogle: String matricula: String	autenticarComToken(token): Boolean
Aluno	Concrete (extends Usuario)	curso: String	—
Professor	Concrete (extends Usuario)	departamento: String	liberarEmprestimo(id): void
Pedagogo	Concrete (extends Usuario)	areaAtuacao: String	liberarEmprestimo(id): void
Diretor	Concrete (extends Usuario)	—	cadastrarUsuario(u): void cadastrarEquipamento(e): void atribuirAutorizacao(email, perfil): void
Equipamento	Concrete	id: Long nome: String codigoPatrimonio: String status: StatusEquipamento	atualizarStatus(novoStatus): void
Emprestimo	Concrete	id: Long dataSolicitacao: DateTime dataDevolucaoPrevista: DateTime status: StatusEmprestimo	—

--	--	--	--

Enumerações

Enumeração	Valores
StatusEquipamento	DISPONIVEL · EMPRESTADO · MANUTENCAO
StatusEmprestimo	PENDENTE · APROVADO · REJEITADO · DEVOLVIDO
TipoPerfil	ALUNO · PROFESSOR · PEDAGOGO · DIRETOR

Relacionamentos

- Usuario (1) ←pertencente_a→ (0..*) Emprestimo: um usuário pode ter múltiplos empréstimos
- Equipamento (1) ←aloca→ (0..*) Emprestimo: um equipamento pode ter múltiplos empréstimos ao longo do tempo
- Usuario (0..1) ←aprovado_por→ (0..*) Emprestimo: o responsável que aprovou o empréstimo

3.4 Diagrama de Sequência

Fluxo: Solicitação de Empréstimo

O diagrama de sequência descreve o fluxo completo de autenticação e criação de solicitação de empréstimo entre os componentes do sistema:

Passo	De	Para	Mensagem
1	Usuário	Frontend	Clica em "Entrar" com credenciais
2	Frontend	Backend (POST /auth/login)	Envia { emailGoogle, senha }
3	Backend	Banco de Dados	Busca usuário pelo e-mail
4	Banco de Dados	Backend	Retorna dados do usuário + senha hash
5	Backend	Backend	Verifica senha com bcrypt.compare()
6	Backend	Frontend	Retorna { access_token, perfil, nome }
7	Frontend	Usuário	Exibe painel conforme o perfil
8	Usuário	Frontend	Seleciona tablet e solicita empréstimo
9	Frontend	Backend (POST /emprestimos)	Envia token JWT + { equipamentoId, datas }
10	Backend	Backend	Valida token JWT e extrai perfil
11	Backend	Banco de Dados	INSERT em emprestimo com status PENDENTE
12	Banco de Dados	Backend	Confirma salvamento
13	Backend	Frontend	Retorna { success: true, id }
14	Frontend	Usuário	Exibe "Aguardando aprovação"

--	--	--	--	--

4. Banco de Dados

4.1 Modelagem Lógica

O banco de dados utiliza o modelo relacional implementado em MySQL, com o nome DB_SISTEM_ESTOQUE. A modelagem segue o padrão de herança de tabela por tipo (Table Per Type — TPT), onde a tabela usuario armazena os atributos comuns e tabelas filhas (aluno, professor, pedagogo, diretor) armazenam os atributos específicos de cada perfil, ligadas por chave estrangeira com o mesmo PK.

4.2 Descrição das Tabelas

Tabela: usuario

Coluna	Tipo	Restrições	Descrição
id	INT	PK, AUTO_INCREMENT	Identificador único do usuário
nome	VARCHAR(150)	NOT NULL	Nome completo do usuário
email_google	VARCHAR(150)	NOT NULL, UNIQUE	E-mail institucional (identificador de login)
senha	VARCHAR(255)	NOT NULL	Hash bcrypt da senha
matricula	VARCHAR(50)	NULLABLE	Matrícula escolar (obrigatória para alunos)
tipo_perfil	ENUM('ALUNO','PROFESSOR','PEDAGOGO','DIRETOR')	NOT NULL	Define as permissões do usuário no sistema
data_cadastro	DATETIME	NOT NULL, DEFAULT NOW()	Data e hora do cadastro

Tabela: aluno

Coluna	Tipo	Restrições	Descrição
usuario_id	INT	PK, FK → usuario(id) ON DELETE CASCADE	Referência ao usuário base
curso	VARCHAR(100)	NULLABLE	Curso do aluno (ex: Desenvolvimento de Sistemas)

Tabela: professor

Coluna	Tipo	Restrições	Descrição
usuario_id	INT	PK, FK → usuario(id) ON DELETE CASCADE	Referência ao usuário base
departamento	VARCHAR(100)	NULLABLE	Departamento do professor (ex: Informática)

Tabela: pedagogo

Coluna	Tipo	Restrições	Descrição
usuario_id	INT	PK, FK → usuario(id) ON DELETE CASCADE	Referência ao usuário base
area_atuacao	VARCHAR(100)	NULLABLE	Área de atuação do pedagogo

Tabela: diretor

Coluna	Tipo	Restrições	Descrição
usuario_id	INT	PK, FK → usuario(id) ON DELETE CASCADE	Referência ao usuário base

Tabela: categoria Equipamento

Coluna	Tipo	Restrições	Descrição
id	INT	PK, AUTO_INCREMENT	Identificador único da categoria
nome	VARCHAR(60)	NOT NULL, UNIQUE	Nome da categoria (ex: Tablet, Mouse, Teclado)

Tabela: equipamento

Coluna	Tipo	Restrições	Descrição
id	INT	PK, AUTO_INCREMENT	Identificador único do equipamento
nome	VARCHAR(150)	NOT NULL	Descrição/nome do equipamento
categoria_id	INT	FK → categoria_equipamento(id)	Categoria do item
codigo_patrimonio	VARCHAR(50)	NOT NULL, UNIQUE	Código de patrimônio da escola (ex: PAT-0001)
status	ENUM('DISPONIVEL','EMPRESTADO','MANUTENCAO')	NOT NULL, DEFAULT DISPONIVEL	Status atual do equipamento
cadastrado_por	INT	FK → usuario(id)	Diretor que cadastrou o item
data_cadastro	DATETIME	NOT NULL, DEFAULT NOW()	Data de cadastro
atualizado_por	INT	NULLABLE, FK → usuario(id)	Quem fez a última edição
data_atualizacao	DATETIME	NULLABLE	Data da última edição

Tabela: emprestimo

Coluna	Tipo	Restrições	Descrição
id	INT	PK, AUTO_INCREMENT	Identificador único do empréstimo
data_solicitacao	DATETIME	NOT NULL, DEFAULT NOW()	Momento da solicitação
data_devolucao_prevista	DATETIME	NULLABLE	Data prevista para devolução
data_devolucao_real	DATETIME	NULLABLE	Data real da devolução (preenchida ao devolver)
status	ENUM('PENDENTE','APROVADO','REJEITADO','DEVOLVIDO')	NOT NULL, DEFAULT PENDENTE	Status atual do empréstimo
usuario_id	INT	FK → usuario(id)	Quem fez a solicitação
equipamento_id	INT	FK → equipamento(id)	Equipamento solicitado
aprovado_por	INT	NULLABLE, FK → usuario(id)	Quem aprovou ou rejeitou

4.3 Triggers e Views

Trigger: trg_emprestimo_status

Executado automaticamente APÓS UPDATE na tabela emprestimo. Atualiza o status do equipamento conforme o status do empréstimo muda, eliminando a necessidade de lógica extra na aplicação:

```
AFTER UPDATE ON emprestimo FOR EACH ROW
```

```
IF NEW.status = "APROVADO" THEN
```

```
UPDATE equipamento SET status = "EMPRESTADO" WHERE id = NEW.equipamento_id;

ELSEIF NEW.status IN ("DEVOLVIDO", "REJEITADO") THEN

    UPDATE equipamento SET status = "DISPONIVEL" WHERE id = NEW.equipamento_id;

END IF;
```

View: vw_status_equipamentos

Consulta consolidada que retorna o status de todos os equipamentos com informações de quem está com cada item e quando deve devolver:

```
SELECT e.id, e.nome, c.nome AS categoria, e.codigo_patrimonio,

       e.status, u.nome AS emprestado_para,

       emp.data_devolucao_prevista

FROM equipamento e

JOIN categoria_equipamento c ON c.id = e.categoria_id

LEFT JOIN emprestimo emp ON emp.equipamento_id = e.id AND emp.status = "APROVADO"

LEFT JOIN usuario u ON u.id = emp.usuario_id;
```

5. Arquitetura do Backend

5.1 Tecnologias Utilizadas

Tecnologia	Versão	Função no Projeto
Node.js	20.x LTS	Runtime JavaScript server-side
NestJS	10.x	Framework backend com arquitetura modular e decorators TypeScript
TypeScript	5.x	Tipagem estática, aumenta confiabilidade e legibilidade do código
TypeORM	0.3.x	ORM para comunicação com o banco de dados MySQL
MySQL	8.x	Sistema de gerenciamento de banco de dados relacional
Passport.js	0.6.x	Middleware de autenticação para NestJS
passport-jwt	4.x	Estratégia JWT para validação de tokens de autenticação
bcrypt	5.x	Biblioteca para hash seguro de senhas
class-validator	0.14.x	Validação de DTOs com decorators
class-transformer	0.5.x	Transformação e serialização de objetos

5.2 Estrutura de Módulos

O backend segue a arquitetura modular do NestJS, onde cada domínio do sistema é encapsulado em um módulo independente:

Módulo	Arquivos	Responsabilidade
Auth	auth.module.ts auth.service.ts auth.controller.ts jwt.strategy.ts	Login, geração de token JWT, validação de credenciais
Users	users.module.ts users.service.ts users.controller.ts user.entity.ts	CRUD de usuários, busca por e-mail, criação de perfis
Equipamentos	equipamentos.module.ts equipamentos.service.ts equipamentos.controller.ts equipamento.entity.ts	CRUD de tablets, consulta de status, view do banco
Categorias	categorias.module.ts categorias.service.ts categorias.controller.ts categoria.entity.ts	Gerenciamento de tipos de equipamento
Empréstimos	emprestimos.module.ts emprestimos.service.ts emprestimos.controller.ts emprestimo.entity.ts	Fluxo completo: solicitação, aprovação, rejeição e devolução
Guards	jwt-auth.guard.ts	Proteção de rotas por autenticação e por perfil

	roles.guard.ts	
Decorators	roles.decorator.ts	@Roles() — define quais perfis podem acessar cada rota

5.3 Rotas da API REST

Auth

Método	Rota	Perfil	Descrição
POST	/auth/login	Público	Autentica com e-mail e senha. Retorna JWT + perfil

Usuários

Método	Rota	Perfil	Descrição
POST	/users	Diretor	Cadastra novo usuário no sistema
GET	/users	Diretor / Pedagogo	Lista todos os usuários
GET	/users/:id	Diretor / Pedagogo	Busca usuário específico por ID

Categorias

Método	Rota	Perfil	Descrição
GET	/categorias	Todos (logados)	Lista todas as categorias de equipamento
POST	/categorias	Diretor	Cria nova categoria de equipamento

Equipamentos

Método	Rota	Perfil	Descrição
GET	/equipamentos	Todos (logados)	Lista todos os equipamentos com categoria
GET	/equipamentos/disponiveis	Todos (logados)	Lista equipamentos disponíveis (usa view do banco)
GET	/equipamentos/ocupados	Todos (logados)	Lista equipamentos emprestados com dados do usuário
GET	/equipamentos/:id	Todos (logados)	Busca equipamento específico por ID
POST	/equipamentos	Diretor	Cadastra novo equipamento no laboratório
PATCH	/equipamentos/:id	Diretor	Atualiza dados de um equipamento existente

Empréstimos

Método	Rota	Perfil	Descrição
GET	/emprestimos	Prof / Ped / Diretor	Lista todos os empréstimos com detalhes
GET	/emprestimos/pendentes	Prof / Ped / Diretor	Lista apenas empréstimos com status PENDENTE
POST	/emprestimos	Todos (logados)	Cria nova solicitação de empréstimo (status PENDENTE)
PATCH	/emprestimos/:id/aprovar	Prof / Ped / Diretor	Aprova empréstimo pendente (status → APROVADO)
PATCH	/emprestimos/:id/rejeitar	Prof / Ped / Diretor	Rejeita empréstimo pendente (status → REJEITADO)
PATCH	/emprestimos/:id/devolver	Prof / Ped / Diretor	Registra devolução (status → DEVOLVIDO, data_real preenchida)

5.4 Autenticação e Autorização

Fluxo de Autenticação (JWT)

- O usuário envia e-mail e senha para POST /auth/login
- O AuthService busca o usuário no banco pelo e-mail
- A senha é comparada com o hash armazenado usando bcrypt.compare()
- Se válido, o JwtService gera um token com payload: { sub: id, nome, emailGoogle, tipo_perfil }
- O token é retornado ao frontend com validade de 8 horas
- Nas requisições seguintes, o token é enviado no header: Authorization: Bearer <token>
- O JwtAuthGuard valida o token em cada rota protegida
- O RolesGuard verifica se o tipo_perfil do token tem permissão para a rota

Guards Implementados

Guard	Arquivo	Função
JwtAuthGuard	jwt-auth.guard.ts	Verifica se há um token JWT válido no header da requisição. Rejeita com 401 se ausente ou inválido.
RolesGuard	roles.guard.ts	Verifica se o perfil do usuário logado está na lista de perfis permitidos para aquela rota, definida pelo decorator @Roles().

6. Protótipo das Telas (Figma)

6.1 Fluxo de Navegação

O fluxo principal do sistema parte da tela Home, passa pelo Login e direciona o usuário ao dashboard correspondente ao seu perfil. A partir do dashboard, cada perfil acessa apenas as funcionalidades que lhe são permitidas.

Tela	Acesso	Navega para
Home	Público	Login · Cadastro
Login	Público	Dashboard (conforme perfil)
Cadastro	Público	Login
Dashboard Aluno	Aluno	Tablets · Solicitar · Meus Empréstimos
Dashboard Professor	Professor	Tablets · Solicitar · Aprovar · Devoluções
Dashboard Pedagogo	Pedagogo	Tablets · Aprovar · Devoluções · Usuários · Histórico
Dashboard Diretor	Diretor	Tablets · Aprovar · Devoluções · Histórico · Cad. Tablet · Usuários · Cad. Usuário

6.2 Telas por Perfil

Telas Públicas (sem login)

Tela	Componentes principais	Função
Home	Hero com título, cards de estatísticas, grade animada, botões de ação	Apresenta o sistema, mostra dados do laboratório em tempo real, direciona para Login/Cadastro
Login	Card de vidro (glassmorphism), campos e-mail/senha, seletor de perfil	Autentica o usuário e redireciona para o dashboard correto
Cadastro	Formulário completo, seletor de perfil, campo de matrícula	Permite que novos usuários se cadastrem no sistema

Perfil: Aluno

Tela	Elementos	Ações disponíveis
Dashboard	Cards: Disponíveis, Meus Pendentes, Histórico · Tabela de últimos empréstimos	Ver resumo · Navegar para outras telas
Tablets	Grid de tablets com status (disponível/emprestado) · Filtros	Ver status de cada tablet · Iniciar solicitação
Solicitar Empréstimo	Seletor de tablet disponível · Campos de data · Campo de finalidade	Selecionar tablet · Definir datas · Confirmar
Confirmação	Resumo dos dados da solicitação · Alerta de status pendente	Confirmar envio · Voltar para corrigir
Meus Empréstimos	Tabela com histórico pessoal de empréstimos e status	Acompanhar status das solicitações

Perfil: Professor

Tela	Elementos	Ações disponíveis
Dashboard	Cards: Disponíveis, Emprestados, Pendentes, Atrasados · Tabela de pendentes	Ver resumo · Aprovar diretamente da tabela
Aprovar Solicitações	Abas (Pendentes/Aprovados/Rejeitados/Todos) · Tabela com ações	Aprovar · Rejeitar cada solicitação
Registrar Devolução	Tabela de tablets emprestados com situação (no prazo/atrasado)	Marcar tablet como devolvido
Solicitar Empréstimo	(Mesmo fluxo do Aluno)	Professor também pode solicitar tablets

Perfil: Pedagogo

Tela	Elementos	Ações disponíveis
Dashboard	Anel de radar (% de utilização) · Cards · Tabela de empréstimos recentes	Visão panorâmica do laboratório
Aprovar Solicitações	Mesma tela do Professor	Aprovar · Rejeitar solicitações
Registrar Devolução	Mesma tela do Professor	Registrar devoluções
Usuários	Tabela com todos os usuários e perfis · Alerta de somente visualização	Visualizar usuários (sem poder editar)
Histórico	Tabela com todos os empréstimos do laboratório	Acompanhar histórico completo

Perfil: Diretor

Tela	Elementos	Ações disponíveis
Dashboard	Anel de radar · Cards completos · Tabela com ações diretas	Aprovar/Rejeitar diretamente · Navegar para qualquer tela
Tablets	Tabela com todos os tablets + botão editar + abas por status	Ver todos os tablets · Editar qualquer tablet
Cadastrar Tablet	Formulário com código de patrimônio e status inicial	Adicionar novo tablet ao laboratório
Usuários	Tabela com todos os usuários, perfis e datas · Botão editar	Ver e editar usuários
Cadastrar Usuário	Formulário completo com perfil, matrícula, senha inicial	Adicionar novos usuários ao sistema
Aprovar Solicitações	Mesma tela de Professor/Pedagogo	Aprovar · Rejeitar
Devoluções	Mesma tela de Professor/Pedagogo	Registrar devoluções
Histórico	Mesma tela do Pedagogo	Acompanhar histórico completo

7. Cronograma de Execução

Semana	Atividade	Entregável	Status
1	Levantamento de requisitos e definição do escopo	Documento de requisitos	✓ Concluído
2	Modelagem do sistema — diagramas de caso de uso, classes e sequência	Diagramas UML	✓ Concluído
3	Modelagem e criação do banco de dados MySQL	Schema SQL (DB_SISTEM_ESTOQUE)	✓ Concluído
4	Desenvolvimento do backend — módulos Users e Auth	API de autenticação	✓ Concluído
5	Desenvolvimento do backend — módulos Equipamentos e Categorias	API de equipamentos	Em andamento
6	Desenvolvimento do backend — módulo Empréstimos	API de empréstimos	Em andamento
6	Prototipagem das telas no Figma	Protótipo navegável	✓ Concluído
7	Testes de integração e ajustes	Sistema testado	Pendente
8	Entrega do MVP e apresentação	Documentação + MVP	Pendente

8. Divisão de Responsabilidades

Integrante	Área	Entregas
Kauã Tatim	Design / Prototipagem	Protótipo no Figma · Telas de todos os perfis · Documentação técnica.
Gabriely	Lider / Back-End	Gestão do projeto · Arquitetura e Infraestrutura · Implementação de módulos e Regras de negócio · Interface e Perfis de acesso · Validações e Organização técnica · Requisitos funcionais e não funcionais.
Igor Rodrigues	Pesquisa / Auxílio	Apesar de buscar orientações sobre suas tarefas, apresentava frequentes distrações e falta de vontade, resultando em poucas contribuições que agregassem valor direto ao projeto.
Guilherme	Banco de Dados	Modelagem · Estrutura e Tabelas · Configurações técnicas e conectividade · Validação e integridade.
Jean	Análise	Fluxograma · Diagrama de classe e caso de uso · Documentação de análise
José	Front-End	Foi designado para auxiliar no Figma em realizando algumas alterações, embora os registros indiquem que não houve avanços significativos no projeto a partir de suas ações. Não demonstrou interesse em se aplicar nas atividades do projeto nos períodos finais de desenvolvimento.

9. Glossário

Termo	Definição
API REST	Interface de programação que segue o padrão REST (Representational State Transfer) para comunicação entre sistemas via HTTP
Autenticação	Processo de verificação da identidade de um usuário (quem é)
Autorização	Processo de verificação das permissões de um usuário autenticado (o que pode fazer)
bcrypt	Algoritmo de hash criptográfico usado para armazenar senhas de forma segura e irreversível
DTO	Data Transfer Object — objeto que define a estrutura dos dados trocados entre cliente e servidor
Guard (NestJS)	Classe que determina se uma requisição pode ser processada, baseada em autenticação ou permissões
JWT	JSON Web Token — token criptografado que carrega informações do usuário e é usado para autenticação stateless
MVP	Minimum Viable Product — produto mínimo viável com as funcionalidades essenciais para validar o sistema
NestJS	Framework Node.js para construção de aplicações server-side escaláveis, baseado em TypeScript
ORM	Object-Relational Mapper — biblioteca que mapeia classes TypeScript para tabelas do banco de dados (TypeORM)
RBAC	Role-Based Access Control — controle de acesso baseado em perfis/papéis de usuário

Trigger	Procedimento armazenado no banco de dados executado automaticamente em resposta a eventos (INSERT, UPDATE, DELETE)
TypeScript	Superset tipado do JavaScript que adiciona verificação estática de tipos ao código
View (SQL)	Consulta SQL salva como objeto de banco de dados, que pode ser consultada como uma tabela virtual

10. Referências

- NestJS Documentation — <https://docs.nestjs.com>
- TypeORM Documentation — <https://typeorm.io>
- MySQL 8.0 Reference Manual — <https://dev.mysql.com/doc/refman/8.0/en/>
- JSON Web Tokens — <https://jwt.io/introduction>
- Passport.js Documentation — <https://www.passportjs.org>
- Figma — <https://www.figma.com>
- Git e GitHub — <https://docs.github.com>
- Sommerville, I. Engenharia de Software. 10. ed. Pearson, 2019.
- Pressman, R. S.; Maxim, B. R. Engenharia de Software: Uma Abordagem Profissional. 8. ed. AMGH, 2016.